

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №9
по курсу «Логика и основы алгоритмизации в
инженерных задачах»
на тему «Поиск расстояний в графе»

Выполнил:

студенты группы 24ВВВ2

Бауэр А.А.

Гусев Е.В.

Медведев Е.А.

Принял:

к.т.н. доцент Юрова О.В.

к.т.н. Деев М.В.

Пенза 2025

Название

Поиск расстояний в графе.

Цель работы

Научиться искать расстояние между вершинами в графе.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++
3. *Реализуйте процедуру поиска расстояний для графа, представленного списками смежности.

Задание 2*

1. *Реализуйте процедуру поиска расстояний на основе обхода в глубину
2. *Реализуйте процедуру поиска расстояний на основе обхода в глубину для графа, представленного списками смежности
3. *Оцените время работы реализаций алгоритмов поиска расстояний на основе обхода в глубину и обхода в ширину для графов разных порядков.

Описание метода решения задачи

Метод решения основан на алгоритмах обхода графов в ширину (BFS) и глубину (DFS) с использованием двух различных структур данных для представления графа: матрицы смежности и списков смежности.

Алгоритм BFS реализует классический поиск в ширину с использованием очереди, где обход начинается со стартовой вершины и последовательно охватывает все вершины на текущем уровне расстояния перед переходом к следующим уровням. Для матричного представления поиск смежных вершин требует полного перебора всех вершин в строке матрицы, что дает сложность $O(V)$ для каждой вершины, где V - количество вершин. В случае списков смежности доступ к соседним вершинам осуществляется напрямую, что обеспечивает сложность $O(k)$, где k - количество соседей вершины.

Алгоритм DFS использует стек для обхода в глубину, где происходит движение по одному пути до его полного исследования с последующим возвратом. В матричной реализации проверка соседей также требует полного перебора строки матрицы, тогда как списковая реализация позволяет эффективно итерироваться только по фактическим соседям вершины.

Во всех случаях BFS показывается более лучшее время поиска расстояния в сравнении с DFS.

Листинг

Файл 9LR_FULL.cpp (1 и 2 задание)

```
#include <iostream>
#include <vector>
#include <queue>
#include <stack>
#include <random>
#include <iomanip>
#include <limits>
#include <chrono>
using namespace std;
using namespace std::chrono;
//Класс для представления графа
class Graph {
private:
    vector<vector<int>> adjacencyMatrix; // Матрица смежности графа
    vector<vector<int>> adjacencyList; // Списки смежности графа
    int vertices; // Количество вершин в графе
    //Метод для построения списков смежности на основе матрицы
    смежности
    void buildAdjacencyListFromMatrix() {
        adjacencyList.clear(); // Очищаем существующие списки
        adjacencyList.resize(vertices); // Изменяем размер под
        количество вершин
        //Проходим по всем элементам матрицы смежности
        for (int i = 0; i < vertices; i++) {
            for (int j = 0; j < vertices; j++) {
                //Если есть ребро между вершинами i и j
                if (adjacencyMatrix[i][j] == 1) {
                    //Добавляем вершину j в список смежности вершины
                    adjacencyList[i].push_back(j);
                }
            }
        }
    }
public:
    //Конструктор класса Graph
    Graph(int n) : vertices(n) {
        generateRandomGraph();
    }
    //Метод для генерации случайного неориентированного графа
    void generateRandomGraph() {
        random_device rd;
        mt19937 gen(rd());
        uniform_int_distribution<> dis(0, 1);
        //Инициализация матрицы смежности нулями
        adjacencyMatrix.resize(vertices, vector<int>(vertices, 0));
        //Генерация неориентированного графа
        for (int i = 0; i < vertices; i++) {
            for (int j = i + 1; j < vertices; j++) {
                int edge = dis(gen);
                adjacencyMatrix[i][j] = edge;
                adjacencyMatrix[j][i] = edge;
            }
        }
    }
};
```

```

    }
}
//Построение списков смежности на основе сгенерированной
матрицы
buildAdjacencyListFromMatrix();
}
//Метод для вывода матрицы смежности
void printAdjacencyMatrix() {
    cout << "\nМатрица смежности:\n";
    //Вывод заголовка
    cout << "    ";
    for (int i = 0; i < vertices; i++) {
        cout << setw(3) << i;
    }
    cout << "\n";
    //Вывод самой матрицы
    for (int i = 0; i < vertices; i++) {
        cout << setw(3) << i;
        for (int j = 0; j < vertices; j++) {
            cout << setw(3) << adjacencyMatrix[i][j];
        }
        cout << "\n";
    }
}
//Метод для вывода списков смежности
void printAdjacencyList() {
    cout << "\nСписки смежности:\n";
    for (int i = 0; i < vertices; i++) {
        cout << i << ": ";
        for (int neighbor : adjacencyList[i]) {
            cout << neighbor << " ";
        }
        cout << "\n";
    }
}
//BFS обход в ширину через матрицу смежности
vector<int> bfsDistanceMatrix(int startVertex) {
    //Засаекаем время
    auto start = high_resolution_clock::now();
    //Вектор для хранения расстояний, инициализирован -1
    (непосещенные вершины)
    vector<int> dist(vertices, -1);
    queue<int> q; //Очередь для BFS
    //Устанавливаем расстояние до стартовой вершины 0 и
    добавляем её в очередь
    dist[startVertex] = 0;
    q.push(startVertex);
    //Пока очередь не пуста
    while (!q.empty()) {
        int current = q.front(); //Берем первую вершину из
очереди
        q.pop(); //Удаляем её из очереди
        //Проверяем всех соседей текущей вершины через матрицу
смежности
        for (int i = 0; i < vertices; i++) {
            //Если есть ребро и вершина еще не посещена

```

```

        if (adjacencyMatrix[current][i] == 1 && dist[i] == -
1) {
            dist[i] = dist[current] + 1; // Обновляем
расстояние
            q.push(i); // Добавляем вершину в очередь для
дальнейшего обхода
        }
    }
    //Останавливаем таймер
    auto end = high_resolution_clock::now();
    auto duration = duration_cast<microseconds>(end - start);
    cout << "Время выполнения BFS (матрица): " <<
duration.count() << " мкс\n";
    return dist; //Возвращаем вектор расстояний
}
//BFS через списки смежности
vector<int> bfsDistanceList(int startVertex) {
    auto start = high_resolution_clock::now();
    vector<int> dist(vertices, -1);
    queue<int> q;
    dist[startVertex] = 0;
    q.push(startVertex);
    while (!q.empty()) {
        int current = q.front();
        q.pop();
        //Используем списки смежности для получения соседей
        for (int neighbor : adjacencyList[current]) {
            if (dist[neighbor] == -1) {
                dist[neighbor] = dist[current] + 1;
                q.push(neighbor);
            }
        }
    }
    auto end = high_resolution_clock::now();
    auto duration = duration_cast<microseconds>(end - start);
    cout << "Время выполнения BFS (списки): " <<
duration.count() << " мкс\n";
    return dist;
}
//DFS обход в глубину через матрицу смежности
vector<int> dfsDistanceMatrix(int startVertex) {
    auto start = high_resolution_clock::now();
    vector<int> dist(vertices, -1);
    stack<int> s; // Стек для DFS
    dist[startVertex] = 0;
    s.push(startVertex);
    while (!s.empty()) {
        int current = s.top(); // Берем вершину с вершины стека
        s.pop(); // Удаляем её из стека
        //Проверяем соседей через матрицу смежности
        for (int i = 0; i < vertices; i++) {
            if (adjacencyMatrix[current][i] == 1 && dist[i] == -
1) {
                dist[i] = dist[current] + 1;

```

```

        s.push(i); // Добавляем в стек для дальнейшего
обхода
    }
}
}
auto end = high_resolution_clock::now();
auto duration = duration_cast<microseconds>(end - start);
cout << "Время выполнения DFS (матрица): " <<
duration.count() << " мкс\n";
return dist;
}

//DFS через списки смежности
vector<int> dfsDistanceList(int startVertex) {
    auto start = high_resolution_clock::now();
    vector<int> dist(vertices, -1);
    stack<int> s;
    dist[startVertex] = 0;
    s.push(startVertex);
    while (!s.empty()) {
        int current = s.top();
        s.pop();
        //Используем списки смежности для получения соседей
        for (int neighbor : adjacencyList[current]) {
            if (dist[neighbor] == -1) {
                dist[neighbor] = dist[current] + 1;
                s.push(neighbor);
            }
        }
    }
    auto end = high_resolution_clock::now();
    auto duration = duration_cast<microseconds>(end - start);
    cout << "Время выполнения DFS (списки): " <<
duration.count() << " мкс\n";
    return dist;
}

//Вывод расстояний
void printDistances(const vector<int>& dist, int startVertex) {
    cout << "\nРасстояния от вершины " << startVertex << ":\n";
    for (int i = 0; i < vertices; i++) {
        cout << "Вершина " << i << ": ";
        if (dist[i] == -1) {
            cout << "недостижима";
        }
        else {
            cout << dist[i];
        }
        cout << "\n";
    }
}

};

int main() {
    setlocale(LC_ALL, "Russian");
    int vertices;
    cout << "Введите количество вершин графа: ";
    cin >> vertices;

```

```

    if (vertices <= 0) {
        cout << "Неверное количество вершин! Установлено значение по
умолчанию: 5" << endl;
        vertices = 5;
    }
    //Создание графа с заданным количеством вершин
    Graph graph(vertices);
    int choice;// Переменная для выбора пользователя
    do
    {
        cout << "\n1. Установить количество вершин";
        cout << "\n2. Показать матрицу смежности";
        cout << "\n3. Показать списки смежности";
        cout << "\n4. BFS - расстояния (матрица смежности)";
        cout << "\n5. BFS - расстояния (списки смежности)";
        cout << "\n6. DFS - расстояния (матрица смежности)";
        cout << "\n7. DFS - расстояния (списки смежности)";
        cout << "\n0. Выход";
        cout << "\nВыберите операцию: ";
        cin >> choice;
        switch (choice) {
            case 1: {
                cout << "Введите количество вершин: ";
                cin >> vertices;
                if (vertices > 0) {
                    graph = Graph(vertices);
                }
                else {
                    cout << "Неверное количество вершин!\n";
                }
                break;
            }
            case 2: {
                graph.printAdjacencyMatrix();
                break;
            }
            case 3: {
                graph.printAdjacencyList();
                break;
            }
            case 4: {
                int startVertex;
                cout << "Введите начальную вершину: ";
                cin >> startVertex;
                if (startVertex >= 0 && startVertex < vertices) {
                    vector<int> dist =
graph.bfsDistanceMatrix(startVertex);
                    graph.printDistances(dist, startVertex);
                }
                else {
                    cout << "Неверная вершина!\n";
                }
                break;
            }
            case 5: {
                int startVertex;

```

```

        cout << "Введите начальную вершину: ";
        cin >> startVertex;
        if (startVertex >= 0 && startVertex < vertices) {
            vector<int> dist =
graph.bfsDistanceList(startVertex);
            graph.printDistances(dist, startVertex);
        }
        else {
            cout << "Неверная вершина!\n";
        }
        break;
    }
    case 6: {
        int startVertex;
        cout << "Введите начальную вершину: ";
        cin >> startVertex;
        if (startVertex >= 0 && startVertex < vertices) {
            vector<int> dist =
graph.dfsDistanceMatrix(startVertex);
            graph.printDistances(dist, startVertex);
        }
        else {
            cout << "Неверная вершина!\n";
        }
        break;
    }
    case 7: {
        int startVertex;
        cout << "Введите начальную вершину: ";
        cin >> startVertex;
        if (startVertex >= 0 && startVertex < vertices) {
            vector<int> dist =
graph.dfsDistanceList(startVertex);
            graph.printDistances(dist, startVertex);
        }
        else {
            cout << "Неверная вершина!\n";
        }
        break;
    }
    case 0: {
        break;
    }
    default: {
        cout << "Неверный выбор!\n";
        break;
    }
}
} while (choice != 0);
return 0;
}

```

Текст к программе

Данная программа реализует алгоритмы обхода графов BFS и DFS с использованием двух структур данных: матрицы смежности и списков

смежности. При тестировании программы были выявлены четкие закономерности в производительности. Программа наглядно демонстрирует важность выбора оптимальной структуры данных для работы с графами, где списки смежности обеспечивают более эффективный обход за счет прямого доступа к смежным вершинам без избыточных проверок.

Результаты работы программы

Результаты работы программы показаны на рисунках 2-3.

```

Введите количество вершин графа: 5
1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 3

Матрица смежности:
  0  1  2  3  4
0  0  0  0  1  1
1  0  0  0  1  0
2  0  0  0  0  0
3  1  1  0  0  0
4  1  0  0  0  0

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 4

Списки смежности:
0: 3 4
1: 3
2:
3: 0 1
4: 0
  
```

Рисунок 1 – Создание графа с 5 вершинами

```

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 5
Введите начальную вершину: 0
Время выполнения BFS (матрица): 37 мкс

Расстояния от вершины 0:
Вершина 0: 0
Вершина 1: 2
Вершина 2: недостижима
Вершина 3: 1
Вершина 4: 1

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 6
Введите начальную вершину: 0
Время выполнения BFS (списки): 30 мкс

Расстояния от вершины 0:
Вершина 0: 0
Вершина 1: 2
Вершина 2: недостижима
Вершина 3: 1
Вершина 4: 1

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 7
Введите начальную вершину: 0
Время выполнения DFS (матрица): 34 мкс

Расстояния от вершины 0:
Вершина 0: 0
Вершина 1: 2
Вершина 2: недостижима
Вершина 3: 1
Вершина 4: 1

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 8
Введите начальную вершину: 0
Время выполнения DFS (списки): 29 мкс

Расстояния от вершины 0:
Вершина 0: 0
Вершина 1: 2
Вершина 2: недостижима
Вершина 3: 1
Вершина 4: 1

```

Рисунок 2 – Расчет расстояний от 0 вершины до остальных

```

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 1
Введите количество вершин: 3

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 3

Матрица смежности:
    0  1  2
0  0  0  0
1  0  0  1
2  0  1  0

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 4

Списки смежности:
0:
1: 2
2: 1

```

Рисунок 4 – Изменение числа вершин в графе

```

Введите количество вершин графа: -5
Неверное количество вершин! Установлено значение по умолчанию: 5

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию:

```

Рисунок 5 – Попытка ввода некорректного значения

```

Введите количество вершин графа: 10

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 5
Введите начальную вершину: 0
Время выполнения BFS (матрица): 36 мкс

Расстояния от вершины 0:
Вершина 0: 0
Вершина 1: 1
Вершина 2: 2
Вершина 3: 2
Вершина 4: 2
Вершина 5: 3
Вершина 6: 2
Вершина 7: 2
Вершина 8: 1
Вершина 9: 2

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 7
Введите начальную вершину: 0
Время выполнения DFS (матрица): 45 мкс

Расстояния от вершины 0:
Вершина 0: 0
Вершина 1: 1
Вершина 2: 2
Вершина 3: 2
Вершина 4: 4
Вершина 5: 3
Вершина 6: 3
Вершина 7: 2
Вершина 8: 1

```

Рисунок 6 – Скорость обхода графа с 10 вершинами 2 методами

```

Введите количество вершин графа: 100

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 5
Введите начальную вершину: 0
Время выполнения BFS (матрица): 419 мкс

```

Рисунок 7 – Скорость обхода графа с 100 вершинами BFS

```

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 7
Введите начальную вершину: 0
Время выполнения DFS (матрица): 469 мкс

```

Рисунок 8 – Скорость обхода графа с 100 вершинами DFS

```

Введите количество вершин графа: 1000

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 5
Введите начальную вершину: 0
Время выполнения BFS (матрица): 25435 мкс

```

Рисунок 9 – Скорость обхода графа с 1000 вершинами BFS

```

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 7
Введите начальную вершину: 0
Время выполнения DFS (матрица): 27242 мкс

```

Рисунок 10 – Скорость обхода графа с 1000 вершинами DFS

```

Введите количество вершин графа: 3000
1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 5
Введите начальную вершину: 0
Время выполнения BFS (матрица): 177887 мкс

```

Рисунок 11 – Скорость обхода графа с 3000 вершинами BFS

```

1. Установить количество вершин
2. Сгенерировать новый граф
3. Показать матрицу смежности
4. Показать списки смежности
5. BFS – расстояния (матрица смежности)
6. BFS – расстояния (списки смежности)
7. DFS – расстояния (матрица смежности)
8. DFS – расстояния (списки смежности)
0. Выход
Выберите операцию: 7
Введите начальную вершину: 0
Время выполнения DFS (матрица): 190879 мкс

```

Рисунок 12 – Скорость обхода графа с 3000 вершинами DFS

Выводы

В ходе лабораторной работы было установлено, что алгоритм BFS демонстрирует более высокую эффективность по сравнению с DFS при решении задачи поиска расстояний в графе. BFS обеспечивает систематический обход вершин по уровням, что гарантирует нахождение кратчайшего пути до каждой достижимой вершины. В отличие от него, DFS может исследовать более длинные пути прежде чем вернуться к ближайшим соседям, что делает

его менее оптимальным для данной задачи. При использовании одинаковых структур данных BFS показал сопоставимое или лучшее время выполнения, подтвердив свою целесообразность для задач поиска расстояний. Наилучшие результаты показала комбинация BFS со списками смежности, которая сочетает эффективность алгоритма с оптимальной структурой хранения графа.